

EXPERIMENT DEFINITION

YAML experiments

configs/*.yaml

arl_robot_1.yaml
base: ARL, hover

↑ extends

helix.yaml
min-time helix

↑ extends

helix_moderate.yaml
fixed timing

↑ extends

helix_cascaded_pid.yaml
PID controller

↓

Composition

config/

loader.py: mappings merge, lists replace; a new kind: replaces the whole section; --set k,path=v
schema.py: strict schema v2

↓ config dict

Command line

python -m isaac_drone

run --backend	fly one mission
synthetic	CPU plant
isaacsim	PhysX on uc1
inspect	PhysX properties
validate	config + USD
audit-usd	authored USD
sweep	synthetic grid
compare	multi-run table
summarize	one-run table
plot	one-run PNGs

RUNTIME · BACKEND INTEGRATION

run_experiment() runtime/runner.py

per physics step: prepare_step() → driver.step() → finish_step() → write record → hooks

hooks

PathOverlayHook
3D path overlay

DisplayHook
real-time pacing

VideoHook
multi-camera MP4

MetricsHook
metrics.json

FiguresHook
plots/*.png

hooks only observe: physics, timing and telemetry are unchanged

↓ record

MotionControlLoop runtime/loop.py

Trajectory
trajectories/
hold · helix
 $p_d, \dots, s_d, \psi_d$
reset: min-time plan + feasibility screen

↓ control rate 100 Hz

Controller
control/
geometric · PID
wrench w_d

↓ physics rate 200 Hz

Allocator
allocation.py
bounded, 3rd sets
thrusters f_i

↓

Motor command
 $n_i = \sqrt{f_i/k_{f,i}}$
spin-up envelope
RPS n_{cmd}

Completion
measured dwell

State provider
truth (default)

Power (off)
thrust bounds

Effects (off)

↑ anti-windup

↑ bounds

↑ read_state(t)

↑ reset: $m, J, c, A, f, \bar{f}$

↓ step()

↓ w_{ext}

↓ n_{cmd}

PHYSICS BACKENDS VehicleBackend + SimulationDriver sim/base.py

reset · read_state · mass_properties · motor_parameters · thrust_to_motor_speeds · apply_motor_speeds · telemetry · allocation_matrix_b driver: step · is_running · is_paused

SyntheticBackend synthetic.py
CPU Newton-Euler rigid body (RK4)
deliberately asymmetric test plant,
not ARL data · seconds, local & CI

ARLBackend isaacsim/
Isaac Lab 3.0 + PhysX, ARL-Robot-1 USD
mass, CoM, inertia, geometry from PhysX
thrust applied to each rotor body

↓ Shared motor model actuation.py NativeRpsActuator: native RPS integrators, $f_i = k_{f,i} n_i^2$

offline commands in plain Python: summarize, plot, compare; sweep also files synthetic runs

DATA · ANALYSIS

Telemetry telemetry/

build_basic_record()
ideal, actual, error (ideal – actual)
RunRecorder
JSONL + CSV + field schema, SI

↓ read back

Run directory

runs/<UT>_<cid>/

config.json
resolved config
metadata.json
plan, backend, provenance
telemetry.jsonl
full record per step
basic.csv
flat SI columns
telemetry_schema.json
units, frames, timing
metrics.json
overall j per phase
plots/*.png
ideal vs actual
*.mp4
optional, isaacsim only

↓ read back

Analysis analysis/

metrics.py
one definition of flight metrics

summary.py
per-phase error table

plots.py
figures from basic.csv

compare.py
runs side by side

sweep.py
parallel synthetic grid

Shared contracts core/

SI units · world Z up
quaternion wxyz, body → world
wrenches about the CoM

↓

types.py
VehicleState · Wrench
TrajectorySetpoint
MassProperties

rotations.py
SO(3) maps; $R_d, \Omega_d, \dot{\Omega}_d$
from thrust direction
and heading

validation.py
reject, never repair:
finite values,
exact shapes

params.py
YAML → frozen
dataclass; unknown
keys rejected

registry.py
kind: → factory
controllers,
trajectories